



Vodič kroz arhitekturu aplikacije

Sistem za upravljanje osiguravajućom kućom — klijenti, polise, štete, osoblje i nalozi. Ovaj dokument prolazi kroz svaki sloj aplikacije, objašnjava zašto je tako organizovano i pokazuje stvarni kod, liniju po liniju gde je to bitno.

.NET 8 / WinForms

FluentNHibernate 3.3

Oracle.ManagedDataAccess

Entiteti → Mapiranje → DTO → DTOManager → Forme

SADRŽAJ

1. Pregled slojeva

2. Konekcija i pokretanje

DataLayer.cs

Program.cs

3. Entiteti

Klijent — puna analiza

Nasleđivanje (FizickoLice/PravnoLice)

4. Mapiranje (NHibernate)

ClassMap / SubclassMap

Veze: HasMany, References, HasManyToMany

5. DTO-ovi

6. DTOManager — poslovni sloj

CRUD nad klijentima

7. Nalozi, prijava i lozinke

8. UI sloj (WinForms)

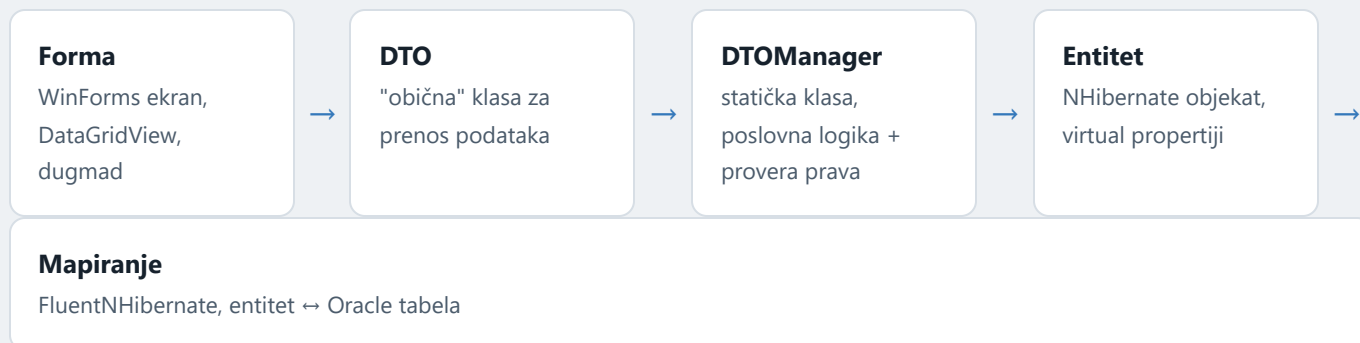
Program → Login → MainForm

KlijentiForma (CRUD ekran)

9. Bezbednosni model

01 Pregled slojeva

Aplikacija je klasična desktop trojka: baza ↔ objekti ↔ ekran, samo razvučena u pet slojeva da bi se poslovna logika i bezbednosne provere držale van formi. Tok podataka kad korisnik npr. doda klijenta izgleda ovako:



Zašto ovoliko slojeva umesto da forma direktno čita iz baze?

ZAŠTO

Entitet (npr. `Klijent`) je "svetinja" NHibernate-a — ima lazy-loading kolekcije, proxy objekte i vezan je za otvorenu sesiju. Kad bi ga forma direktno prikazivala, pucalo bi čim se sesija zatvori. DTO je "mrtva", obična klasa — sigurno se nosi kroz UI, servijalizuje, prikazuje u gridu, bez ikakve veze sa bazom.

02 Konekcija i pokretanje aplikacije

Sve počinje od dva mala fajla: `DataLayer.cs` (ko drži konekciju) i `Program.cs` (šta se prvo pokreće).

DataLayer.cs — fabrika sesija

```
OsiguranjeApp/DataLayer.cs
```

```
1 public class DataLayer
2 {
```

```

3 private static ISessionFactory? _factory = null;
4 private static readonly object _lock = new object();
5 public static ISession GetSession()
6 {
7     if (_factory == null)
8     {
9         lock (_lock)
10        {
11            if (_factory == null)
12                _factory = CreateSessionFactory();
13        }
14    }
15    return _factory.OpenSession();
16 }

```

```
private static ISessionFactory? _factory = null;
```

je teška, skupa stvar — čita mapiranja, konektuje se samo za život aplikacije, samo jednom na Oracle, gradi keš zato je metapodataka. Pravi se

```
if (_factory == null) { lock(_lock) { if (_factory == null) ... } }
```

— klasičan obrazac za "lenjo" (lazy) kreiranje jedinog objekta koji sme da postoji u više niti bez nepotrebnog zaključavanja pri svakom pozivu. WinForms je jednonitna aplikacija pa ovde nije kritično, ali je bezopasan defanzivni kod.

```
return _factory.OpenSession();
```

(jedinic / unit-c work). P je jedna sesije se otvaraju Svaki `GetSession()` vraćanovu `ISession` zatvara operaci videćeš obrazac svakoj

CreateSessionFactory() – konfiguracija

```
1 var cfg = OracleManagedDataClientConfiguration.Oracle10
2     .ConnectionString(c => c.Is(
3         "Data Source=gislab-oracle.elfak.ni.ac.rs:1521/SBP_PDB;" +
4         "User Id=S19451;Password=S19451;"));
5 return Fluently.Configure()
6     .Database(cfg)
7     .Mappings(m => m.FluentMappings
8         .AddFromAssemblyOf<KlijentMapiranje>())
9     .BuildSessionFactory();
```

- `OracleManagedDataClientConfiguration.Oracle10` — kaže FluentNHibernate da generiše Oracle-kompatibilan SQL (npr. sekvence umesto auto-increment kolona, `ROWNUM` stil paginacije itd).
- `.AddFromAssemblyOf<KlijentMapiranje>()` — umesto da se svako mapiranje ručno registruje, FluentNHibernate refleksijom pokupi **sve** klase u istom assembly-ju koje nasleđuju `ClassMap<T>` ili `SubclassMap<T>`. Zato dodavanje novog fajla u `Mapiranja/` automatski "uđe u igru" bez ijedne dodatne linije konfiguracije.

NAPOMENA

Konekcioni string (korisnik/lozinka) je zakucan direktno u kodu. Radi za studentski projekat na fakultetskoj bazi, ali u pravoj aplikaciji bi ovo išlo u konfiguracioni fajl van git repozitorijuma.

Program.cs — ulazna tačka

OsiguranjeApp/Program.cs

```
1 [STAThread]
2 static void Main()
3 {
4     Application.EnableVisualStyles();
5     Application.SetHighDpiMode(HighDpiMode.SystemAware);
6
7     bool nastavi = true;
8     while (nastavi)
9     {
10         using var login = new LoginForma();
```

```

11         if (login.ShowDialog() != DialogResult.OK || !SesijaKorisnik.JeUlogovan)
12             return;
13         SesijaKorisnik.ZahtevZaOdjavu = false;
14         Application.Run(new MainForm());
15         nastavi = SesijaKorisnik.ZahtevZaOdjavu;
16     }
17 }
18 }

```

SUŠTINA

[STAThread] je obavezno za WinForms (COM zahteva single-threaded apartment). Petlja `while(nastavi)` je "prijavi se → radi → ako se korisnik odjavio, vrati na login" — umesto `Application.Exit()`, MainForm samo postavi `SesijaKorisnik.ZahtevZaOdjavu = true` i zatvori se; petlja to pročita i ponovo prikaže `LoginForma`. Ako korisnik na loginu klikne "Izlaz" ili X, `ShowDialog()` ne vrati `OK` i `Main` se prosto završi.

03 Entiteti — kako izgleda "objekat = red u tabeli"

Entiteti su u fascikli `Entiteti/`. Ovo je najprostiji i najbrojniji sloj — svaka klasa je gotovo 1:1 slika jedne tabele. Umesto da prolazim kroz svih 32, uzmimo `Klijent` kao potpun primer, pa objasnim šta se ponavlja svuda ostalo.

Entiteti/Klijent.cs

```

1  public class Klijent
2  {
3      public virtual int KlijentId { get; set; }
4      public virtual string? Naziv { get; set; }
5      public virtual string? Adresa { get; set; }
6      public virtual string? Telefon { get; set; }
7      public virtual string? Email { get; set; }
8      public virtual string? Status { get; set; }
9      public virtual DateTime DatumRegistracije { get; set; }
10     public virtual string? TipKlijenta { get; set; }
11     public virtual IList<Polisa> Polise { get; set; }
12     public virtual IList<Steta> Stete { get; set; }
13     public Klijent()
14     {
15         Polise = new List<Polisa>();
16         Stete = new List<Steta>();
17     }
18 }

```

```

20 public override string ToString() => Naziv ?? "";
21 }

```

```
public virtual int KlijentId { get; set; }
```

(dinamički generisane potklase pr Castle DynamicPro da bi omogućio lenjo učitavanje (lazy loading) — a proxy može da "presretnu" pristup property-ju samo ako je virtuelan i može da se override-uj

Svaki property koji NHibernate treba da mapira mora biti

NHibernate **virtual** NHibernateproxy u pozadini klase pravi

```
string? Naziv, Adresa, ... - nullable
```

Projekat ima

<Nullable>enable</Nullable> .csproj- u, pa je

```
ICollection<Polisa> Polise / ICollection<Steta> Stete
```

Kolekcije predstavljaju "jedan-naspram-više" vezu (jedan klijent ima više polisa/šteta). NHibernate ih pri čitanju iz baze puni

se izvrši tek kad se kolekcija stvarno pročita, ne pri učitavanju klijenta.

— lenjoSQL Polise za

```
public Klijent() { Polise = new List<Polisa>(); ... }
```

Kolekcije se (još kolekciju inicijalizuju u nesačuvani) novi null — inače novi

Nasleđivanje: FizickoLice / PravnoLice / JavnaInstitucija

Klijent u stvarnom svetu nije samo jedna vrsta — može biti fizičko lice, pravno lice ili javna institucija, svako sa svojim dodatnim poljima. Umesto tri nepovezane klase, kod koristi pravo OOP nasleđivanje:

```
1 public class FizickoLice : Klijent
2 {
3     public virtual string? Jmbg { get; set; }
4     public virtual DateTime? DatumRodjenja { get; set; }
5     public virtual string? Zanimanje { get; set; }
6 }
7
8 public class PravnoLice : Klijent
9 {
10    public virtual string? Pib { get; set; }
11    public virtual string? MaticniBroj { get; set; }
12    public virtual string? Delatnost { get; set; }
13    public virtual IList<KontaktOsoba> KontaktOsobe { get; set; }
14 }
15 public class JavnaInstitucija : Klijent // napomena: NE nasleđuje PravnoLice, iako liči
16 {
17    public virtual string? Pib { get; set; }
18    public virtual string? MaticniBroj { get; set; }
19    public virtual string? Delatnost { get; set; }
20    public virtual string? NivoInstitucije { get; set; }
21 }
22 }
```

DETALJ KOJI SE LAKO PREVIDI

`JavnaInstitucija` ima ista polja kao `PravnoLice` (Pib, MaticniBroj, Delatnost) plus `NivoInstitucije`, ali **ne nasleđuje** `PravnoLice` — nasleđuje direktno `Klijent`. To je namerno ponavljanje polja umesto dubljeg stabla nasleđivanja, verovatno da bi mapiranje ostalo jednostavno (jedan nivo `SubclassMap`, umesto lanca podklasa).

04 Mapiranja — most između klase i tabele

Fascikla **Mapiranja/** sadrži **FluentNHibernate** mape: C# kod (umesto starinskog XML-a) koji kaže NHibernate-u tačno koja kolona ide na koji property. Ovo je najgušći deo aplikacije konceptualno, pa idemo baš liniju po liniju.

Mapiranja/KlijentMapiranje.cs

```
1 public class KlijentMapiranje : ClassMap<Klijent>
2 {
3     public KlijentMapiranje()
4     {
5         Table("KLIJENT");
6         Id(x => x.KlijentId).Column("KLIJENT_ID").GeneratedBy.Sequence("KLIJENT_ID_SEQ");
7         Map(x => x.Naziv).Column("NAZIV").Not.Nullable();
8         Map(x => x.Adresa).Column("ADRESA");
9         Map(x => x.Telefon).Column("TELEFON");
10        Map(x => x.Email).Column("EMAIL");
11        Map(x => x.Status).Column("STATUS");
12        Map(x => x.DatumRegistracije).Column("DATUM_REGISTRACIJE");
13        Map(x => x.TipKlijenta).Column("TIP_KLIJENTA").Not.Nullable();
14        HasMany(x => x.Polise).KeyColumn("UGOVARAC_ID").Cascade.None();
15        HasMany(x => x.Stete).KeyColumn("PODNOSILAC_ID").Cascade.None();
16    }
17 }
```

class KlijentMapiranje : ClassMap<Klijent>

Konvencija
FluentNHibernate:
jedna klasa po
entitetu,
nasleđuje
generički

ClassMap<T>

. Sva
"konfiguracija"
se piše u
konstruktoru
— fluent-style
pozivi metoda
koji se
ulančavaju.

Table("KLIJENT");

Ime tabele u Oracle šemi. Bez ovoga
NHibernate bi po konvenciji pokušao da
pogodi ime iz naziva klase.

Id(x =>
x.KlijentId).Column("KLIJENT_ID").GeneratedBy.
Sequence("KLIJENT_ID_SEQ")

Primarni
ključ.

GeneratedBy.Sequence(...)

govori:
Oracle
nema
auto-
increment
kolone
kao

MySQL,
pa se
vrednost
povlači iz
eksplicitne

```
Map(x =>  
x.Naziv).Column("NAZIV").Not.Nullable();
```

Obično je
skalarno `Not.Nullable()` NHibernate-informacija
polje. ova

```
HasMany(x =>  
x.Polise).KeyColumn("UGOVARAC_ID").Cascade.Non  
e();
```

Kaže:
"u `POLISA` postoji `UGOVARAC_ID` ka ovom `Polis`
tabeli kolona
koja je
strani
ključ
nazad
klijentu
— to je
moja
kolekcija

SubclassMap — nasleđivanje u bazi (table-per-subclass)

Mapiranje/FizickoLiceMapiranje.cs

```
1 public class FizickoLiceMapiranje : SubclassMap<FizickoLice>  
2 {  
3     public FizickoLiceMapiranje()  
4     {  
5         Table("FIZICKO_LICE");
```

```

6      KeyColumn("KLIJENT_ID");
7      Map(x => x.Jmbg).Column("JMBG");
8      Map(x => x.DatumRodjenja).Column("DATUM_RODZENJA");
9      Map(x => x.Zanimanje).Column("ZANIMANJE");
10   }
11 }

```

Ovo je strategija **"table per subclass"** (poznata i kao joined-subclass): bazna tabela `KLIJENT` drži zajednička polja, a `FIZICKO_LICE` drži *samo* dodatna polja i ima `KLIJENT_ID` koji je istovremeno i primarni ključ i strani ključ nazad na `KLIJENT`. Kad NHibernate učita klijenta, po potrebi **spoji (JOIN)** obe tabele i vrati ti pravu C# klasu (`FizickoLice` ili `PravnoLice`) — zato u `DTOManager.Klijenti.cs` radi `if (k is FizickoLice fl)`: NHibernate ti je već vratio konkretan tip, ne generičkog `Klijent`-a.

Sve vrste veza u aplikaciji

Iste četiri-pet FluentNHibernate metode se ponavljaju kroz svih ~30 mapiranja. Evo šta svaka znači i gde se koristi:

Poziv	Značenje	Primer iz koda
<code>References(x)</code>	Više-naspram-jedan (FK ka drugoj tabeli, npr. strani ključ koji pokazuje na jedan red)	<code>Polisa.Agent,</code> <code>Polisa.Ugovarac,</code> <code>Nalog.Osoblje</code>
<code>HasMany(x)</code>	Jedan-naspram-više (kolekcija koju drži "strana jedan")	<code>Klijent.Polise,</code> <code>Polisa.Stete,</code> <code>Osoblje.FazeObrade</code>
<code>HasManyToMany(x)</code>	Više-naspram-više preko posredničke tabele	<code>AutoOsiguranje.Vozaci</code> ↔ tabela <code>VOZAC_POLISE</code>
<code>Cascade.None()</code>	Snimanje/brisanje "vlasnika" se NE prenosi na povezane redove	<code>Klijent</code> → <code>Polise</code> , <code>Polisa</code> → <code>Stete</code>
<code>Cascade.AllDeleteOrphan()</code>	Povezani redovi žive i umiru sa vlasnikom — pravo "kompozicija" vezivanje	<code>Polisa</code> → <code>DodatnaPokrića,</code> <code>PravnoLice</code> → <code>KontaktOsobe</code>
<code>.Inverse()</code>	"Ova strana veze ne upravlja FK kolonom" — sprečava duple UPDATE upite kad oba kraja veze znaju jedno za drugo	<code>Polisa.DodatnaPokrića,</code> <code>Polisa.Istorija</code>
<code>SubclassMap<T></code>	Nasleđivanje entiteta preko JOIN-a dve tabele	<code>FizickoLice,</code> <code>AutoOsiguranje,</code> <code>Agent,</code> <code>AutoSteta</code> ...

```

1 public class AutoOsiguranjeMapiranje : SubclassMap<AutoOsiguranje>
2 {
3     public AutoOsiguranjeMapiranje()
4     {
5         Table("AUTO_OSIGURANJE");
6         KeyColumn("POLISA_ID");
7         References(x => x.Vozilo).Column("VOZILO_ID");
8         Map(x => x.BonusMalusKlasa).Column("BONUS_MALUS_KLASA");
9         HasManyToMany(x => x.Vozaci)
10             .Table("VOZAC_POLISE")
11             .ParentKeyColumn("POLISA_ID")
12             .ChildKeyColumn("KLIJENT_ID")
13             .Cascade.None();
14     }
15 }

```

Auto osiguranje (podklasa polise) referencira jedno **Vozilo** (References — više polisa mogu deliti isto vozilo istorijski, ali svaka polisa ima tačno jedno), i ima listu vozača (**Vozaci**) koji su **Klijent** objekti — jedan vozač može biti naveden na više polisa i jedna polisa može imati više vozača, otud posrednička tabela **VOZAC_POLISE**.

05 DTO-ovi — šta forma stvarno vidi

DTO (Data Transfer Object) klase su u fascikli **DTOs/**. Za razliku od entiteta, ovo su **obične** klase — bez **virtual**, bez veze sa NHibernate sesijom, slobodne da žive koliko god dugo je formi potrebno.

DTOs/KlijentDTO.cs

```

1 public class KlijentPregled
2 {
3     public int      KlijentId      { get; set; }
4     public string?  Naziv          { get; set; }
5     public string?  TipKlijenta    { get; set; }
6     // ... Adresa, Telefon, Email, Status, DatumRegistracije
7     public override string ToString() => Naziv ?? "";
8 }
9
10 public class KlijentBasic : KlijentPregled
11 {
12     public IList<PolisaPregled> Polise { get; set; }
13     public IList<StetaPregled> Stete  { get; set; }

```

```

14 }
15 public class FizickoLicePregled : KlientPregled
16 {
17     public string? Jmbg { get; set; }
18     public DateTime? DatumRodjenja { get; set; }
19     public string? Zanimanje { get; set; }
20 }
21 }

```

Ovde postoje dve namerne konvencije koje se provlače kroz **ceo** projekat:

Sufiks	Namena
... Pregled	"Laka" verzija za liste / DataGridView — samo skalarna polja, bez ugnjeđenih kolekcija. Koristi se za pretragu i tabelarni prikaz gde bi učitavanje svih povezanih polisa/šteta za stotine redova bilo skupo i nepotrebno.
... Basic	"Puna" verzija za ekran detalja jednog zapisa — nasleđuje Pregled i dodaje kolekcije povezanih objekata (npr. <code>KlijentBasic</code> nosi i listu polisa i šteta tog klijenta).

Nasleđivanje DTO-a **oslikava** nasleđivanje entiteta (`FizickoLicePregled` : `KlijentPregled` , baš kao `FizickoLice : Klient`) — ali su to dve potpuno odvojene hijerarhije koje slučajno imaju isti oblik. Nema nikakvog automatskog mapiranja između njih (ni AutoMapper-a ni sličnog alata); svako polje se ručno prepisuje u DTOManager-u.

ZAŠTO SE POLJA RUČNO PREPISUJU UMETO AUTOMATSKOG MAPERA

U ovolikoj bazi (32 entiteta) automatski mapper bi uštedeo linije koda, ali bi sakrio tačno mesto gde se poslovna pravila mogu primeniti (npr. `Status = dto.Status ?? "AKTIVAN"` ispod). Ručno mapiranje je "glupo" ali eksplicitno — lako je pratiti šta odakle dolazi.

06 DTOManager — poslovni sloj

Ovo je srce aplikacije. `DTOManager` je **jedna** logička klasa razbijena kroz više fajlova pomoću C# ključne reči `partial` : `DTOManager.Klijenti.cs` , `DTOManager.Polise.cs` , `DTOManager.Stete.cs` , `DTOManager.Osoblje.cs` , `DTOManager.Nalog.cs` , `DTOManager.Autorizacija.cs` — svi definišu deliće iste klase `public partial class DTOManager` , samo grupisano po domenu radi preglednosti fajlova. Svaka metoda je `static` — forme je pozivaju direktno kao `DTOManager.vratiSveKlijente()` , bez instanciranja ičega.

Standardni obrazac jedne metode

DTOManager.Klijenti.cs – dodajFizickoLice

```
1 public static void dodajFizickoLice(FizickoLiceBasic dto)
2 {
3     ProveriOvlascenje("ADMIN", "AGENT");
4     try
5     {
6         ISession s = DataLayer.GetSession();
7         var fl = new FizickoLice
8         {
9             Naziv = dto.Naziv, Adresa = dto.Adresa,
10            Status = dto.Status ?? "AKTIVAN",
11            DatumRegistracije = DateTime.Today,
12            TipKlijenta = "FIZICKO_LICE",
13            Jmbg = dto.Jmbg, DatumRodjenja = dto.DatumRodjenja
14        };
15        s.Save(fl);
16        s.Flush();
17        s.Close();
18    }
19    catch (Exception ex) { MessageBox.Show(ex.Message, "Greška"); }
20 }
```

ProveriOvlascenje("ADMIN", "AGENT");

Prva linija
svake
metode
koja
menja
podatke.
Definisana
u

DTOManager.Autorizacija.cs

baca

Ne

ISession s = DataLayer.GetSession(); ...
s.Close();

bloka. To znači da
NHibernate sesija
živi tačno onoliko
koliko traje jedna

	Sesija zatvara operacija (učitaj-ili-se ručno unutar using snimi), što je otvara, iste namerno: sprečava koristi metode "curenje" i — otvorenih nema konekcija i izbegava probleme sa lazy-loading van sesije.
<pre>s.Save(fl); s.Flush();</pre>	insert u NHibernate-ovoj Save() samo zakažuje internoj listi Flush() baza promena (persistence context); SQL eksp
<pre>catch (Exception ex) { MessageBox.Show(...); }</pre>	Svaka metoda guta izuzetke i prikazuje ih direktno korisniku kroz . Praktično za desktop aplikaciju ovog obima — korisnik odmah vidi šta je MessageBox pošlo naopako (npr. constraint violation iz Oracle-a) bez potrebe za logovanjem/telemetrijom.

Mapiranje entiteta u DTO — **is** pattern matching

```

1 private static KlientPregled mapKlientPregled(Klient k)
2 {
3     if (k is FizickoLice fl)
4         return new FizickoLicePregled { /* zajednička polja + */ Jmbg = fl.Jmbg, ... };
5     if (k is PravnoLice pl)
6         return new PravnoLicePregled { ... Pib = pl.Pib, ... };
7     if (k is JavnaInstitucija ji)
8         return new JavnaInstitucijaPregled { ... NivoInstitucije = ji.NivoInstitucije };
9     return new KlientPregled { /* samo bazna polja */ };
10 }
```

Zato što je mapiranje `Klijent` tabele **table-per-subclass**, NHibernate vraća pravi runtime tip (`FizickoLice`, ne generički `Klijent`). C# `is` pattern matching to iskorišćava da izabere odgovarajući DTO. Ovaj tačan obrazac (privatna `mapXPregled` metoda pozvana iz svih javnih "vrati/pretraži" metoda) ponavlja se u `DTOManager.Polise.cs` i `DTOManager.Stete.cs`.

PREGLED SVIH JAVNIH OPERACIJA NAD KLIJENTIMA

Metoda	Šta radi	Ko sme
<code>vratiSveKlijente()</code>	Cela lista, sortirana po nazivu	svi ulogovani
<code>pretraziKlijente(naziv, tip)</code>	LINQ filter nad nazivom (Contains) i tipom	svi ulogovani
<code>vratiKlijenta(id)</code>	<code>s.Load<Klijent>(id)</code> → puni <code>KlijentBasic</code>	svi ulogovani
<code>dodajFizickoLice / dodajPravnoLice / dodajJavnuInstituciju</code>	Kreira konkretnu podklasu i snima	ADMIN, AGENT
<code>azurirajKlijenta(dto)</code>	Učita entitet, prepíše polja, <code>SaveOrUpdate</code>	ADMIN, AGENT
<code>obrisiKlijenta(id)</code>	<code>s.Delete(k)</code>	samo ADMIN

LOAD VS GET

Kod svuda koristi `s.Load<T>(id)` a ne `s.Get<T>(id)`. `Load` vraća **proxy** odmah bez upita ka bazi (pretpostavlja da red postoji) i puca tek kad se prvi put pristupi property-ju ako reda nema; `Get` odmah upita bazu i vrati `null` ako ne postoji. Pošto se ovde ID uvek uzima iz reda koji je korisnik već video u gridu, `Load` je bezbedan i brži izbor.

07 Nalozi, prijava i lozinke — najsloženiji deo aplikacije

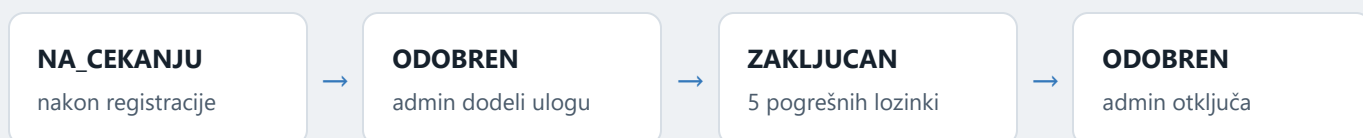
Ovo je jedini deo domena koji nije prosto CRUD — ima stanja, radni tok (workflow) i kriptografiju. Vredi ga proći detaljno.

Dva entiteta

Entitet	Predstavlja	Bitna polja
Nalog	Korisnički nalog za prijavu (1 zaposleni = najviše 1 aktivan/na čekanju nalog)	KorisnickoIme, LozinkaHash, LozinkaSalt, Uloga, StatusNaloga, NeuspesnihPrijava
IstorijaPrijave	Audit log — svaki pokušaj prijave, uspešan ili ne	Nalog, KorisnickoImePokusaj, VremePokusaja, Uspesno, Razlog

Mašina stanja naloga

`StatusNaloga` ide kroz jasno definisana stanja, svako sa svojim dozvoljenim prelazima — ovo je logika koju metode u `DTOManager.Nalog.cs` striktno čuvaju (npr. `odobriNalog` odbija sve što nije `NA_CEKANJU`):



Ili grana `NA_CEKANJU → ODBIJEN` (admin odbije zahtev; sledeći put isti zaposleni pokuša registraciju, stari `ODBIJEN` zapis se briše).

Heš lozinke — PBKDF2

DTOManager.Nalog.cs — HashLozinku / ProveriLozinku

```

1 private const int PbkdfIteracije = 100000;
2 private static (string hash, string salt) HashLozinku(string lozinka)
4 {
5     byte[] saltBytes = RandomNumberGenerator.GetBytes(16);
6     using var pbkdf2 = new Rfc2898DeriveBytes(lozinka, saltBytes, PbkdfIteracije, HashAlgo
7     byte[] hashBytes = pbkdf2.GetBytes(32);
8     return (Convert.ToBase64String(hashBytes), Convert.ToBase64String(saltBytes));
9 }
10 private static bool ProveriLozinku(string lozinka, string hash, string salt, int iteracije
12 {
13     byte[] saltBytes = Convert.FromBase64String(salt);
14     using var pbkdf2 = new Rfc2898DeriveBytes(lozinka, saltBytes, iteracije, HashAlgorith
15     byte[] hashBytes = pbkdf2.GetBytes(32);
16     byte[] ocekivano = Convert.FromBase64String(hash);
17     return CryptographicOperations.FixedTimeEquals(hashBytes, ocekivano);
18 }
  
```



```
RandomNumberGenerator.GetBytes(16)
```

Za svaki
nalog se **jedinstvena**
generiše

nasumična "so" (salt) od 16
bajtova. Bez soli, dve
osobe sa istom lozinkom
bi imale identičan heš u
bazi — što otkriva
podudaranje i olakšava
napade rainbow-table
tabelama.

```
Rfc2898DeriveBytes(lozinka, salt, 100000,  
SHA256)
```

PBKDF2
— **spor**
namerno

algoritam (100.000 iteracija SHA-
256) baš zato da brute-force
napad na ukradenu bazu bude
praktično neizvodljiv. Obična
SHA-256 lozinke direktno bila bi
loša praksa jer je prebrza
(milijarde pokušaja u sekundi na
GPU).

```
CryptographicOperations.FixedTimeEquals( ... )
```

Poređenje **konstantnom** —
heševa u **vremenu**

običan



nad bajt
nizovima
prekida
poređenje
čim naiđe na
prvu razliku,
što
napadaču
kroz
merenje
vremena
odgovora
(timing
attack)
može odati
koliko su
prvi bajtovi
tačni. Ova
metoda
uvek troši
isto vreme
bez obzira
gde je
razlika.

Tok prijave (prijavaSe)

Metoda `prijavaSe(korisnickoIme, lozinka)` je niz uzastopnih provera, svaka sa sopstvenom porukom i sopstvenim upisom u `IstorijaPrijave` preko pomoćne `ZabelezPokusaj(...)`:

1. Nalog ne postoji → beleži `NEPOSTOJECI_NALOG`, generička poruka (namerno ista poruka kao za pogrešnu lozinku — sprečava da napadač otkrije koja korisnička imena postoje).
2. Status `ZAKLJUCAN` → odbija odmah, bez provere lozinke.
3. Status `NA_CEKANJU` ili `ODBIJEN` → odbija sa objašnjenjem statusa.
4. Lozinka ne odgovara → `NeuspesnihPrijava++`; ako pređe `MaxNeuspesnihPrijava = 5`, status automatski postaje `ZAKLJUCAN`.
5. Sve prođe → resetuje brojač, upiše `ZadnjaPrijava = DateTime.Now`, vrati `NalogPregled` upakovan u `PrijavaRezultat`.

Promena lozinke — self-service vs. admin

```
1 public static (bool uspeh, string poruka) promeniLozinku(int nalogId, string staraLozinka,
2 {
3     bool jeAdminZaTudjiNalog = SesijaKorisnik.ImaUlogu("ADMIN") && SesijaKorisnik.Trenutni
4     if (!jeAdminZaTudjiNalog)
5         ProveriOvlascenje("ADMIN", "AGENT", "LEKAR", "PRAVNIK", "PROCENITELJ");
6     // ... validacija dužine/cifre ...
7
8     if (!jeAdminZaTudjiNalog)
9     {
10         bool ispravna = ProveriLozinku(staraLozinka, n.LozinkaHash!, n.LozinkaSalt!, n.Lo
11         if (!ispravna) return (false, "Stara lozinka nije ispravna.");
12     }
13     // ... upiše novi hash/salt ...
14 }
```

Jedna metoda pokriva dva slučaja: korisnik menja **svoju** lozinku (mora uneti staru — provereno) ili admin resetuje lozinku **drugom** nalogu (ne zna i ne mora znati staru, preskače tu proveru). Razlika se svodi na jedno bulovo `jeAdminZaTudjiNalog` izračunato na početku, koje utiče i na to koja provera prava se poziva i da li se stara lozinka uopšte proverava.

`resetujLozinku(nalogId, privremenaLozinka)` (samo ADMIN) postavlja `MoraPromenitiLozinku = true` i po potrebi otključava nalog — tipičan "zaboravio sam lozinku" tok, ali u ovoj verziji koda nijedna forma trenutno ne proverava `MoraPromenitiLozinku` da bi prisilila promenu na sledećoj prijavi; polje postoji u modelu i bazi, spremno za tu proveru.

08 UI sloj — WinForms

Fascikla `Forme/`. Za razliku od tipičnog Visual Studio Windows Forms projekta, ovde **nema** Designer.cs fajlova za svaku formu — kontrole se prave ručno u kodu preko pomoćne klase `UiHelper` (mada neke novije forme, poput `KlijentiForma`, ipak imaju `.Designer.cs` parnjak generisan iz VS dizajnera). Oba stila žive u istom projektu.

SesijaKorisnik — jedini deljeni state

```
1 public static class SesijaKorisnik
2 {
3     public static NalogPregled? TrenutniNalog { get; set; }
4     public static bool ZahtevZaOdjavu { get; set; }
5     public static bool JeUlogovan => TrenutniNalog != null;
6     public static bool ImaUlogu(params string[] uloge) =>
7         TrenutniNalog != null && uloge.Contains(TrenutniNalog.Uloga);
8     public static void Odjava() => TrenutniNalog = null;
9 }
```

Statička klasa koja drži "ko je trenutno ulogovan" — jednostavna zamena za pravi sesijski mehanizam jer je ovo desktop aplikacija sa jednim korisnikom po pokretanju procesa.

`ImaUlogu("ADMIN", "AGENT")` se poziva i u formama (da se dugmad sakriju) i u `DTOManager` (kroz `ProveriOvlascenje`) — **ista** logika, dva mesta primene, jedno za estetiku, jedno za stvarnu bezbednost.

LoginForma — ručno građen UI

```
1 var naslov = UiHelper.NapraviNaslov("🔒 Prijava u sistem");
2 var tbl = UiHelper.NapraviLayout(3);
3 txtKorisnickoIme = UiHelper.DodajRed(tbl, 0, "Korisničko ime:");
4 txtLozinka       = UiHelper.DodajRed(tbl, 1, "Lozinka:");
```

```

5 txtLozinka.UseSystemPasswordChar = true;
6 var (btnOk, btnCancel) = UiHelper.DodajDugmadPanel(tbl, 2, "✓ Prijavi se", "✕ Izlaz");

```

UiHelper (u **Forme/UiHelper.cs**) je fabrika UI komponenti koja garantuje da svaki ekran izgleda dosledno bez copy-paste stilizovanja: **NapraviNaslov** pravi plavu traku sa belim naslovom (boja **Plava = RGB(30,64,103)** — ista nijansa koju vidiš na vrhu ove stranice, pozajmljena baš odatle), **NapraviLayout** pravi dvokolonski **TableLayoutPanel** (labela | polje), **DodajRed/DodajComboRed/DodajDTPRed** ubacuju jedan red forme (label + TextBox/ComboBox/DateTimePicker), a **StilizirajGrid** boji **DataGridView** zaglavlje u istu plavu, sa naizmeničnim redovima.

Program → LoginForma → MainForm

MainForm je **MDI roditelj** (Multiple Document Interface) — sve funkcionalne forme (KlijentiForma, PoliseForma, SteteForma...) se otvaraju kao njegova "deca":

```

1 private void OtvoriFormu(Form forma)
2 {
3     foreach (Form f in this.MdiChildren)
4         if (f.GetType() == forma.GetType())
5             { f.Activate(); forma.Dispose(); return; }
6     forma.MdiParent = this;
7     forma.WindowState = FormWindowState.Maximized;
8     forma.Show();
9 }
10

```

Pre otvaranja nove forme, provjerava da li ista vrsta forme već postoji među MDI decom — ako da, samo je fokusira umesto da otvori duplikat (npr. dupli klik na "Klijenti" ne otvara dva prozora sa listom klijenata).

PrimeniUlogu() se poziva iz konstruktora MainForm-a i sakriva dugmad **btnOsoblje** / **btnNalozi** ako ulogovani nije ADMIN — isti obrazac "sakrij dugme" viđen i u **KlijentiForma** (gde LEKAR/PRAVNIK/PROCENITELJ dobijaju samo-za-čitanje režim).

KlijentiForma — anatomija jednog CRUD ekrana

```

1 private void ucitajKlijente()
2 {
3     var lista = DTOManager.pretraziKlijente(txtPretraga.Text.Trim(), cmbTip.SelectedItem?.
4     dgvKlijenti.SelectionChanged -= dgvKlijenti_SelectionChanged;
5     dgvKlijenti.Rows.Clear();
6

```

```

7      foreach (var k in lista)
8      {
9          int r = dgvKlijenti.Rows.Add(k.KlijentId, k.Naziv, ...);
10         dgvKlijenti.Rows[r].Cells["colStatus"].Style.ForeColor = UiHelper.StatusBoja(k.Sta
11         dgvKlijenti.Rows[r].Tag = k;
12     }
13     dgvKlijenti.SelectionChanged += dgvKlijenti_SelectionChanged;
14 }

```

```

dgvKlijenti.SelectionChanged -= ...; ... +=
...;

```

pre
punjenja
grida i
vrati
Handler se otkrači Rows.Add(...) okinulo
privremeno posle —
bez
ovoga
bi svako

```

dgvKlijenti.Rows[r].Tag = k;

```

svakog
Ceo reda
DTO grid. samo
objekat Tag Kad odabraniKlijent() pročit
se čuva korisnik
u klikne
red,

```

UiHelper.StatusBoja(k.Status)

```

Mala funkcija koja mapira string status
("AKTIVAN", "BLOKIRAN"...) u boju
(zelena/crvena/narandžasta) — vizuelni signal
statusa bez ikakve dodatne ikonice, korišćen
dosledno u svim listama (klijenti, polise, štete).

```

this.Load += (s, e) => this.BeginInvoke(new
Action(ucitajKlijente));

```

što
for
pot



umesto ubacuje prik
učitavanje spre
BeginInvoke direktnog Load podataka posleda s
poziva u u red inic
poruka upi
(message baz
queue) blo
da se sam
izvrši iscr
pro

Brisanje ide preko `UiHelper.PokusajAkciju()` ⇒ `DTOManager.obrisiKlijenta(k.KlijentId))` — pomoćna funkcija koja hvata baš `NeovlascenPristupException` (razlikuje "nemaš pravo" od ostalih grešaka) i prikaže lepu poruku "Pristup odbijen" umesto sirovog teksta izuzetka.

09 Bezbednosni model — sažetak

Uloga	Tipično može
ADMIN	Sve — jedini koji upravlja nalogima, osobljem, brisanjem klijenata
AGENT	Dodaje/menja klijente, polise, prijavljuje štete
LEKAR / PRAVNIK / PROCENITELJ	Samo pregled klijenata i polisa; rade obradu šteta u svojoj specijalnosti
NEODOBREN	Dodeljuje se pri registraciji dok admin ne odobri nalog — praktično bez prava

Provera prolazi kroz dve linije odbrane koje uvek dolaze zajedno u paru:

1. **UI nivo** — forma sakriva/onemogućava dugmad koja korisnik ionako ne sme koristiti (kozmetika, sprečava zbunjenost, ne bezbednost).
2. **DTOManager nivo** — `ProveriOvlascenje( ... )` na početku svake metode koja menja podatke baca `NeovlascenPristupException` ako uloga ne odgovara. **Ovo je stvarna brava** — čak i kad bi neko zaobišao UI (npr. pozvao metodu direktno), server-side (ovde: DTOManager-side) provera i dalje stoji.

ZAŠTO JE OVO BITNO U DESKTOP APLIKACIJI

Pošto forme i DTOManager žive u **istom** procesu (nema pravog servera između), granica "klijent/server" je ovde arhitekturna, ne mrežna. Provera u DTOManager-

u je i dalje vredna jer sprečava greške (dugme se zaboravi sakriti u novoj formi) i jasno dokumentuje ko šta sme, na jednom mestu.

10 Mapa celog domena

Isti obrazac (Entitet + Mapiranje + DTO + deo DTOManager-a) se ponavlja za svaku od ovih grupa. Umesto da ponavljam identično objašnjenje, evo šta svaka grupa modeluje:

Klijenti

Klasa	Uloga
Klijent baza	Zajednička polja: naziv, adresa, kontakt, status, tip
FizickoLice podklasa	Fizičko lice — JMBG, datum rođenja, zanimanje
PravnoLice podklasa	Firma — PIB, matični broj, delatnost, ima KontaktOsobe
JavnaInstitucija podklasa	Kao PravnoLice + NivelInstitucije (npr. republički/lokalni)
KontaktOsoba vezana	Kontakt osoba u ime pravnog lica/institucije

Polise (osiguranja)

Klasa	Uloga
Polisa baza	Broj polise, period važenja, premija, ugovarač (Klijent), agent
AutoOsiguranje podklasa	Vozilo, bonus-malus klasa, lista vozača (many-to-many)
ZivotnoOsiguranje podklasa	Ima Korisniksplate (korisnici osiguranja i % udela)
ZdravstvenoOsiguranje podklasa	Zdravstveno osiguranje lica
PutnoOsiguranje podklasa	Putno osiguranje
ImovinskOsiguranje podklasa	Nekretnina kao osigurani objekat
DodatnoPokrice / IstorijaPolise / UlogaKlijenta vezane	Dodatna pokrića, istorija izmena statusa, uloga klijenta na polisi (npr. suvozač, korisnik)
Vozilo / Nekretnina osigurani objekat	Predmet osiguranja vezan za konkretnu podklasu polise

Štete

Klasa	Uloga
Steta baza	Prijava štete vezana za Polisu i Klijenta (podnosioca)
AutoSteta / ZdravstvenaSteta / ImovinskSteta podklase	Specifična polja po vrsti štete
FazaObrade vezana	Koraci obrade štete kroz vreme, sa odgovornim licem (Osoblje)
ProcenaStete vezana	Procena procenitelja — nalaz, iznos, preporuka
OsteceniPredmet / OstecenLice vezane	Šta je oštećeno / ko je povređen u okviru štete

Osoblje i nalozi

Klasa	Uloga
Osoblje baza	Zaposleni — ime, JMBG, tip osoblja, datum angažovanja
Agent podklasa	Prodaje polise — licenca, region, % provizije
Lekar podklasa	Obrađuje zdravstvene štete — specijalizacija
Pravnik podklasa	Pravna obrada predmeta
Procenitelj podklasa	Procenjuje štete u svojim oblastima (OblastProc)
Nalog vezana	Nalog za prijavu vezan za jedno Osoblje (1:1 kroz workflow odobravanja)
IstorijaPrijave vezana	Audit log pokušaja prijave

Za svaku od gornjih grupa fajlovi se zovu isto po obrascu: `Entiteti/Ime.cs`, `Mapiranja/ImeMapiranje.cs`, `DTOs/ImeDTO.cs`, i metode negde unutar odgovarajućeg `DTOManager.*.cs` fajla — ako umeš da čitaš `Klijent` lanac iz ovog dokumenta, isti "recept" važi i za ostatak.

